

CRAP·mutation과 기능 해결의 상관관계

유효 실행 / 계획	기능 해결	조사 · 작성
54 / 54회	27 / 54회	항화인
누적 에이전트 작업 시간	그중 기록된 공개 검사 시간	모델 요청
64시간 21분 48초	18시간 51분 51초	12,886건
유효 54회의 개별 작업 시간 합 · 3회 병렬 실행	공식 호출 97회 중 완료된 95회 · 직접 측정 시간은 별도 집계되지 않음	최종 54회 기준 · 제외 · 중단된 7시도까지 포함하면 14,699건

공개 검사 시간은 작업 시간에 포함되므로 두 시간을 더하지 않는다. 작업 시간은 에이전트 시작부터 종료까지이며, 종료 후 비공개 채점 시간은 제외한다.

실험 요약

CRAP은 코드의 복잡성과 테스트가 실행한 범위를 합친 위험값이다. 낮을수록 좋다. mutation은 코드에 일부러 넣은 작은 오류를 테스트가 찾아낸 비율로, 높을수록 좋다. Prime Agent(model: deepseek-v4-flash)로 결함 3개를 고치도록 하고, 품질 목표를 주지 않은 A와 목표를 준 B~F를 비교했다. 각 그룹은 9회씩 실행했다.

기능 해결은 A 4/9회, B·C·E 5/9회, D·F 4/9회였다. C는 모든 실행에서 CRAP 8 이하와 mutation 90% 이상을 기록했지만 기능 해결은 5회였다. 품질 수치를 만족하는 것과 요구된 동작을 완성하는 것은 별개의 결과다.

품질 목표를 받은 그룹은 A보다 CRAP이 낮고 mutation 점수가 높았다. 기능 해결은 그룹마다 9회 중 4~5회로, 한 회 차이였다. 티켓별로는 VD-971이 18회 모두 해결됐고, VD-984는 18회 모두 미해결로 기록됐으며, VD-994는 18회 중 9회 해결됐다.

품질 수치와 기능 해결을 나누어 읽는다

그룹	지시한 목표	기능 해결	CRAP 위험값 ¹	mutation 평균(%)	요청/실행	분/실행
A	목표 없음	4/9	76.08	51.7	103.2	13.9
B	CRAP 8 · mutation 80%	5/9	7.74	95.1	246.3	84.1
C	CRAP 8 · mutation 90%	5/9	6.56	97.4	271.3	79.3
D	CRAP 8 · mutation 100%	4/9	9.40	100.0	266.4	70.5
E	CRAP 6 · mutation 90%	5/9	5.44	98.3	284.0	102.6
F	CRAP 10 · mutation 90%	4/9	15.59	95.2	260.4	78.6

표 1. 각 그룹 9회. ¹CRAP 위험값은 실행마다 가장 높은 함수 값을 고른 뒤 9개를 평균했다(5쪽 설명). mutation은 실행별 검출률의 평균이다. 기능 해결은 결함 테스트를 모두 통과한 실행 수다.

A는 실행당 평균 13.9분, 품질 목표를 받은 B~F는 평균 83.0분을 썼다. 품질 지표는 좋아졌지만 실행 시간도 늘었다. 시간에는 코드 수정뿐 아니라 에이전트가 직접 실행한 검사와 대기가 포함된다.

읽는 기준: 기능 해결은 비공개 결함 테스트로 판단한다. CRAP과 mutation은 수정 코드와 테스트의 품질을 별도로 보여준다. 품질 점수가 높다는 이유만으로 기능이 완성됐다고 판단하지 않는다.

1. 여섯 그룹은 무엇이 다른가

같은 모델과 티켓을 사용하고, 마지막 목표 지시를 그룹별로 달리했다. A도 테스트 작성 지시는 받았다.

그룹	CRAP 최대	mutation 최소	모델이 받은 지시의 차이
A	수치 없음	수치 없음	티켓 전체 해결, 회귀 테스트 작성, 관련 테스트 실행과 확인. CRAP·mutation이라는 단어와 검사 도구 이름을 지시에 넣지 않았다.
B	8 이하	80%	CRAP은 8 이하, mutation은 80% 이상을 목표로 지시했다.
C	8 이하	90%	CRAP은 8 이하, mutation은 90% 이상을 목표로 지시했다.
D	8 이하	100%	mutation 목표를 100%로 높였다. 모든 실행 가능한 변종을 찾아내야 한다.
E	6 이하	90%	CRAP 상한을 6으로 낮춰 코드 위험값을 더 엄격하게 제한했다.
F	10 이하	90%	CRAP 상한을 10으로 높여 코드 위험값의 허용 범위를 넓혔다.

표 2. 그룹별 목표. 검사 도구는 모든 그룹의 환경에 설치했으며, A에게는 도구 이름과 사용법을 알려주지 않았다.

목표 지시문은 세 조각을 이어 붙였다

티켓 원문 전체 + 공통 작업 지시 + 그룹 지시	
① 티켓	에이전트에게 결함 티켓의 제목, 재현 절차, 현재 동작과 기대 동작을 전달했다. 같은 티켓을 수행하는 실행에는 모두 같은 원문을 사용했다.
② 공통	"위 티켓의 모든 요구사항을 준수해 결함을 수정하라. 관련 회귀 테스트를 작성하고 실행하며, 기존 관련 테스트도 실행하고 결과를 확인하라. 단계별 사용자 승인을 요청하거나 기다리지 말고, 안전한 범위에서 합리적으로 가정에 수정과 검증을 끝까지 진행하라." 여섯 그룹 모두 같은 바이트다.
③ 그룹	C의 예: "prime-quality-check --crap-max 8 --mutation-min 90을 실행해 변경 함수의 CRAP와 mutation을 확인하라. 부족하면 수정과 테스트를 보강한 뒤 재측정하라. 변경 함수의 CRAP는 8 이하여야 하고 mutation은 90% 이상이어야 한다." 그룹별로 달라지며 A에는 이 부분이 생략된다.

세 조각을 이은 지시문을 Prime Agent의 지속 목표(--goal)로 등록했다. 지속 목표는 에이전트가 응답을 마쳐도 목표가 완료 상태가 아니면 작업을 다시 이어가게 하는 기능이다. 이렇게 다시 이어간 횟수를 이 보고서에서 목표 이어가기 횟수라고 부른다. 시작 문구는 "등록된 목표 전체를 수행하고, 완료 조건을 충족하면 goal.complete()를 호출하세요."로 모든 그룹에 같았다. 자율 실행 옵션은 어느 그룹에도 켜지 않았다.

티켓 3개와 실행 54회

티켓	고쳐야 하는 동작	결함 테스트	기존 테스트
VD-971(쉬움)	첨부 파일 여러 개 중 하나가 용량 상한을 넘으면 앞서 올린 파일까지 저장소에 남지 않게 할 것	4개	412개
VD-994(중간)	빈 목록의 페이지 번호 보정, 답변 동시 등록 시 409 응답, 첨부 삭제 실패의 재시도와 후속 처리 3건	5개	416개
VD-984(어려움)	인증키를 다시 등록할 때 복구할 모델만 켜고, 사용자가 직접 끈 모델은 유지하며, 두 번 등록해도 두 번 켜지지 않을 것	5개	425개

표 3. 난이도 명칭은 이 실험의 해결 결과에 따른 설명용 구분이다. 결함 테스트는 수정 전에 실패하고 수정 후에 통과해야 하는 비공개 테스트이고, 기존 테스트는 원래 정상인 기능을 다시 검사한다. 대상 저장소는 사내 백엔드 저장소 하나다.

고정한 것	값
모델 · 접속	deepseek-v4-flash · 지정 HTTP 엔드포인트 1곳 · context window 131,072 토큰
하네스	Prime Agent 한 종류. 코드 버전과 실행 파일을 고정해 54회 모두 같은 것을 썼고, 기본 스킬을 모든 그룹에 허용
규모와 순서	6그룹 × 3티켓 × 3회 = 54회. 각 그룹이 같은 티켓을 3번씩 수행했다. 특정 그룹이 늘 먼저 실행되지 않도록 순서를 번갈아 배치했다.
한도	실행당 외부 시간 한도 14,400초. 모델 요청 수 상한 없음. 동시에 3회 실행

표 4. 실행 조건. 시간 한도에 닿은 실행은 무효가 아니라 목표 미완료 결과로 남긴다.

2. 어떻게 격리하고, 어떻게 채점했나

공식 공개 검사와 최종 품질 채점은 같은 계산을 별도 사본에서 수행한다. 에이전트가 직접 만든 측정 코드는 공식 검사와 구분한다.

① 실행 54회는 서로 보지 못한다

분리 영역	구현 방법	효과
파일·설정	실행마다 작업 폴더, 설정 폴더(HOME), 세션 폴더, 임시 폴더를 새로 만들었다.	다른 실행의 코드·기록·검사 결과를 읽을 수 없다.
프로그램·네트워크	운영체제가 실행마다 보이는 사용자, 파일 경로, 프로그램 목록, 네트워크, 프로그램 간 통신을 따로 나누는 기능(네임스페이스)을 실행마다 새로 만들었다.	다른 실행의 프로그램을 보거나 간섭할 수 없고, 모델 엔드포인트 외에는 접속할 수 없다.
품질 측정	공식 중계 서버를 통한 공개 검사와 최종 채점은 테스트 수집·실행, 커버리지, 각 변종 측정마다 새 작업 사본과 새 격리 공간을 사용했다.	한 변종으로 인한 부작용이 다음 결과에 남지 않는다.

표 5. CPU, 메모리, 모델 서버는 세 실행이 공유했다. 작업 내용의 격리이지 응답 시간의 격리는 아니다.

② 에이전트가 보는 검사와 보고서의 채점은 목적이 다르다

검사	누가 언제	무엇을
공개 검사 prime-quality-check	에이전트가 원할 때 직접 실행. B~F는 지시로, A는 자발적으로 실행할 수 있었다.	에이전트가 추가·수정한 테스트 파일을 실행해, 수정 코드의 CRAP과 mutation을 계산한다. 도구는 결과를 에이전트에게 보여준다. 비공개 정답 테스트는 사용하지 않는다.
비공개 채점 hidden grader	에이전트 프로세스가 끝난 뒤 실행기가 별도 사본에서 실행.	채점기는 비공개 정답 테스트와 기존 테스트로 기능 해결을 판정한다. 품질 측정에서는 에이전트가 추가·수정한 테스트 파일을 따로 실행해 CRAP과 mutation을 계산한다. 결과는 에이전트에게 돌려주지 않는다.

표 6. 공식 공개 검사는 작업 중 품질 수치를 확인한다. 에이전트가 별도로 만든 측정은 공식 기록에 잡히지 않을 수 있다. 기능 해결 여부는 종료 후 비공개 결함 테스트로 판정한다.

③ 결과 확정 규칙

작업 종료 → 사본 보존 → 기능 판정 → 품질 판정 → 배정 그룹 유지	
보존	에이전트 프로세스가 끝나면 실행기가 작업 폴더를 통째로 보존하고, 무시 대상 파일과 빈 폴더를 걸어낸 정규 사본을 만든다. 공개 검사와 비공개 채점은 이 정규 사본의 별도 복사본을 쓴다.
판정	기능 해결은 결함 테스트 전부 통과, 기존 테스트 신규 실패 0건, 기존 테스트 소실 0건일 때만 인정한다. 이 보고서는 기능 해결, CRAP, mutation을 각각 나누어 보여준다.
비교	배정받은 그룹을 기준으로 집계한다. 공개 검사 도구를 실행하지 않았거나 목표 수치에 못 미친 실행도 해당 그룹에 포함한다. 같은 티켓내에서 반복해서 얻은 결과를 대조했다.

3. 축 A · 기능 해결

아래 수치는 비공개 테스트가 기록한 판정이다.

해결 = 결함 테스트 전부 통과 + 기존 정상 테스트의 신규 실패 0건 + 기존 테스트 소실 0건. 부분 점수는 없다. 54회 전체에서 신규 실패와 소실은 0건이었으므로, 이 실험의 미해결은 모두 결함 테스트 미통과다.

그룹	VD-971(쉬움)	VD-984(어려움)	VD-994(중간)	합계	해결률 %
A	3/3	0/3	1/3	4/9	44.4
B	3/3	0/3	2/3	5/9	55.6
C	3/3	0/3	2/3	5/9	55.6
D	3/3	0/3	1/3	4/9	44.4
E	3/3	0/3	2/3	5/9	55.6
F	3/3	0/3	1/3	4/9	44.4
B~F 합	15/15	0/15	8/15	23/45	51.1

표 7. 당시 채점 기록. 각 그룹은 티켓마다 3회씩, 총 9회 실행했다. VD-984의 분모는 6그룹 × 3회 = 18회이며 54회 전체가 아니다.

VD-971은 18회 모두 해결, VD-994는 18회 중 9회 해결, VD-984는 18회 모두 미해결로 기록됐다.

어떤 요구사항에서 막혔나

티켓	결함 테스트가 확인하는 동작	A 실패	B~F 실패
VD-984(어려움)	한 번 복구한 뒤 사용자가 끈 모델은 다음 키 등록에서도 꺼진 채로 남는다	2/3	15/15
VD-984(어려움)	키 삭제로 자동 해제된 모델만 복구하고, 사용자가 꺼 둔 모델은 유지한다	1/3	7/15
VD-984(어려움)	다른 모델의 기록이 많아도 복구 대상을 찾아낸다	1/3	7/15
VD-984(어려움)	모델 전용 키를 등록해도 공급사 전체 키의 삭제 기록으로 모델을 켜지 않는다	3/3	4/15
VD-984(어려움)	기록이 없으면 복구하지 않는다	0/3	1/15
VD-994(중간)	저장소 삭제가 계속 실패하면 요청이 실패한다	2/3	7/15
VD-994(중간)	저장소 삭제가 재시도로 성공하면 요청이 이어진다	2/3	7/15
VD-994(중간)	답변 동시 등록 경합도 409로 끝난다	0/3	1/15
VD-994(중간)	빈 목록의 범위 밖 페이지는 1로 보정된다	0/3	0/15
VD-994(중간)	삭제 실패는 지워진 것으로 기록되지 않는다	0/3	0/15

표 8. 칸은 그 테스트에 실패한 실행 수/실행 수다. 쉬운 티켓의 결함 테스트 4개는 18회 모두 통과해 표에서 뺐다.

4. 측 B · CRAP

CRAP은 함수의 복잡도와 테스트가 실행한 코드의 비율로 계산한다. 복잡도가 높거나 테스트가 실행하지 않은 부분이 많으면 값이 커진다. 채점기는 한 실행에서 가장 높은 함수 CRAP을 대표값으로 기록했다.

$CRAP = \text{복잡도}^2 \times (1 - \text{커버리지})^3 + \text{복잡도}$

복잡도는 코드의 실행 경로가 갈라지는 정도다. 커버리지는 채점기가 테스트를 돌렸을 때 해당 함수에서 실행된 줄의 비율이다. 식에는 100%를 1로 넣는다. 커버리지가 100%이면 CRAP은 복잡도와 같고, 복잡도 2인 함수의 커버리지가 0%이면 CRAP은 6이다.

채점기는 에이전트가 추가하거나 수정한 함수마다 CRAP을 계산하고, 그중 가장 높은 값 하나를 골랐다. 표의 ‘9회 평균’은 같은 그룹의 실행 9회에서 고른 값들을 평균한 것이다. 모든 함수 값을 평균하면 위험한 함수의 높은 값이 다른 함수의 낮은 값에 가려질 수 있어, 실행별 최댓값을 사용했다.

그룹	CRAP 최댓값 9회 평균	CRAP 최댓값 9회 중간값	모든 수정 함수 CRAP 8 이하	모든 수정 함수 커버리지 100%	수정 함수 수 실행당 평균
A	76.08	15.57	1/9	0/9	5.9
B	7.74	6.00	8/9	4/9	8.3
C	6.56	7.00	9/9	8/9	6.9
D	9.40	7.12	7/9	5/9	7.3
E	5.44	5.00	9/9	7/9	8.2
F	15.59	8.07	4/9	5/9	6.4

표 9. 각 그룹 9회. 중간값은 대표값 9개를 작은 순서로 놓았을 때 다섯 번째 값이다. ‘모든 수정 함수’ 열은 조건을 만족한 실행 수/9회다. A는 큰 값 두 개(254.20, 324.68) 때문에 평균 76.08이 중간값 15.57보다 높다.

채점기는 어떤 테스트로 커버리지를 측정했나

에이전트의 작업이 끝나면 채점기는 에이전트가 추가하거나 수정한 테스트 파일을 실행했다. 채점기는 이 테스트들이 수정된 함수의 어느 줄을 실행했는지 기록했다. 비공개 정답 테스트는 이 커버리지 측정에 넣지 않았다.

채점기는 A그룹의 9회 모두를 측정했다. 각 실행에서는 수정 함수 중 적어도 하나에, 이 테스트들이 실행하지 않은 코드 줄이 남았다. 반면 B~F의 45회 중 29회에서는 채점기가 돌린 테스트들이 모든 수정 함수의 측정 대상 줄을 빠짐없이 실행했다.

에이전트가 작업 중 공개 품질 검사 도구를 호출했는지는 별도로 집계했다. 또한 테스트가 코드 줄을 실행했더라도, 그 줄의 동작이 요구사항에 맞는지까지 검증했다는 뜻은 아니다.

CRAP이 낮아도 요구사항을 놓칠 수 있다

CRAP은 에이전트가 추가하거나 수정한 함수만 평가한다. 에이전트가 필요한 기능을 빠뜨리거나 요구사항과 다른 동작을 구현해도, 코드가 단순하고 커버리지가 높으면 CRAP은 낮게 나올 수 있다.

5. 측 C · mutation

에이전트가 쓴 테스트가 코드의 작은 오류를 잡는지 본다. 높을수록 좋고, 그룹 목표는 정확한 비율이 하한 이상일 때만 달성이다.

mutation 점수 = 검출한 변종 수 ÷ 실행 가능한 변종 수 × 100

채점기는 수정한 함수의 연산자나 상수를 한 곳 바꾼 코드 사본(변종)을 만든다. 채점기가 각 사본에서 에이전트의 추가·수정 테스트 파일을 실행했을 때 테스트가 실패하면 오류를 검출한 것이다. 테스트가 그대로 통과하면 검출하지 못한 것이다. 시간 초과나 실행 오류가 있으면 완전한 점수로 기록하지 않는다.

그룹	점수 평균	중앙값	90% 이상	검출/대상 평균	테스트 수 평균	등록된 공개 검사 완료/요청	공식 검사 기록이 있는 실행
A	51.7	55.2	0/8	61.0/116.3	22.4	0/0	0/9
B	95.1	98.0	7/9	89.4/93.8	38.6	23/24	9/9
C	97.4	98.5	9/9	79.3/82.4	40.3	21/21	5/9
D	100.0	100.0	9/9	84.7/84.7	39.0	16/16	5/9
E	98.3	100.0	9/9	77.3/79.1	41.8	16/17	7/9
F	95.2	95.2	9/9	89.3/94.2	43.2	19/19	7/9

표 10. 점수는 실행별 검출률의 평균이다. A만 8회 측정: 나머지 1회는 변종 79개 중 1개가 시간 초과로 끝났다. 검출/대상은 실행별 변종 수의 평균이므로 두 평균을 나누는 값과 mutation 점수 평균은 다를 수 있다. 공식 검사 횟수에는 측정 기능을 직접 호출한 경우가 포함되지 않는다.

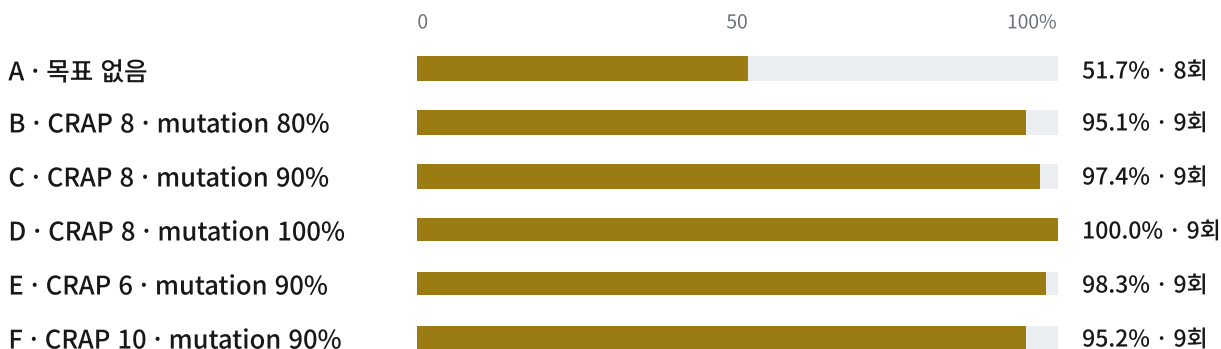


그림 1. 그룹별 mutation 점수 평균. A는 8회, B~F는 각 9회 측정값이다.

B부터 F 45회 중 43회가 90% 이상이었고, B의 2회만 80%대였다. D는 9회 모두 정확히 100%였다. A는 8회 중 90% 이상이 없었으며 최고 65.8%, 최저 16.7%였다. 테스트 수도 A 평균 22.4개에서 B~F 평균 40.6개로 늘었다.

이 측이 재지 않는 것

채점기는 에이전트가 수정한 코드와 추가·수정한 테스트 파일로 mutation을 측정했다. 에이전트가 요구사항을 잘못 이해해 코드와 테스트를 같은 방향으로 틀리게 작성하면, mutation 점수가 높아도 실제 기능은 잘못 동작할 수 있다.

6. 두 지표와 기능 해결의 상관관계

품질 수치와 당시 기능 판정이 어떻게 함께 나타났는지 비교했다.

① 기준을 만족한 실행은 얼마나 해결됐나

기준	기준 만족: 해결 수/실행 수	기준 미달: 해결 수/실행 수	Fisher p
CRAP 8 이하	23/38 (60.5%)	4/16 (25.0%)	0.035
mutation 90% 이상	21/43 (48.8%)	5/10 (50.0%)	1.000

표 11. CRAP 행은 54회, mutation 행은 측정값이 있는 53회다. 예를 들어 CRAP 8 이하인 38회 중 23회가 기능을 해결했다. 각 지표를 따로 비교한 결과다.

Fisher p는 두 집단의 실제 해결률이 같다고 가정했을 때, 지금 관측한 차이와 같거나 더 큰 차이가 나올 확률이다. 0~1 사이이며 값이 클수록 지금 정도의 차이는 해결률이 같아도 흔히 나올 수 있다는 뜻이다.

CRAP 기준을 만족한 실행은 60.5%, 만족하지 못한 실행은 25.0%가 기능을 해결했다. $p=0.035$ 이므로, 실제 해결률이 같다면 이 정도 이상의 차이가 나올 확률은 3.5%다. mutation은 48.8%와 50.0%로 거의 같고 $p=1.000$ 이므로, 두 집단의 해결률이 다르다고 볼 근거가 없었다.

② 같은 티켓 안에서도 그런가

티켓	CRAP 8 이하의 해결	CRAP 8 초과와 해결	mutation 90% 이상의 해결	mutation 90% 미만의 해결
VD-971(쉬움)	15/15	3/3	14/14	3/3
VD-984(어려움)	0/9	0/9	0/15	0/3
VD-994(중간)	8/14	1/4	7/14	2/4

표 12. VD-971은 18회 모두 해결, VD-984는 18회 모두 미해결이라 두 티켓 안에서는 해결 여부와의 상관을 계산할 수 없다. VD-994의 CRAP 기준 차이는 표본이 작아 불확실하다(Fisher $p=0.576$). mutation의 p 는 1.000이다.

③ 순위 상관계수

지표	전체 실행의 순위 상관	이 실험에서 읽는 방향
CRAP 위험값	-0.350	값이 낮은 쪽에서 기능 해결이 더 자주 나타났다.
mutation %	+0.133	점수가 높은 쪽과 기능 해결의 연결이 약했다.

표 13. CRAP 54회, mutation 53회. 순위 상관은 수치의 크기 자체보다 순서를 비교한다. 극단적인 이상치가 있어도 값이 왜곡되지 않는다.

0과 가까울수록 순위와 관련 없다는 것이며 +1로 갈수록 두 값의 순서가 양의 상관, -1로 갈수록 음의 상관이다. (CRAP 은 낮을수록 좋고, Mutation 은 높을수록 좋은 값이다.)

7. 작업 시간과 결과의 해석

품질 목표를 받은 B~F는 A보다 모델 요청이 평균 2.6배, 작업 시간이 약 6배였다. 여기의 비용은 청구 금액이 아니라 요청 수와 걸린 시간이다.

그룹	모델 요청 / 실행	작업 시간 / 실행(분)	기록된 공개 검사 시간(분) 작업 시간 중 비율
A	103.2	13.9	0.0 (0.0%)
B	246.3	84.1	38.2 (45.4%)
C	271.3	79.3	16.9 (21.3%)
D	266.4	70.5	13.0 (18.4%)
E	284.0	102.6	24.6 (23.9%)
F	260.4	78.6	33.1 (42.1%)

표 14. 각 그룹 9회의 평균. 공개 검사 시간은 공식 중계 서버에 기록된 검사 중 완료된 검사의 시간이며, 작업 시간에 포함된다. 에이전트가 직접 실행한 개별 테스트나 자체 CRAP·mutation 측정, 종료 후 채점기의 검사 시간은 이 열에 포함하지 않았다.

검사와 대기를 포함한 시간이다

B는 실행당 84.1분 중 공식 공개 검사에 38.2분, F는 78.6분 중 33.1분을 썼다. mutation 검사는 코드에 넣은 오류마다 테스트를 반복하므로 시간이 든다. 직접 실행한 측정과 자원 대기도 작업 시간에 포함된다.

세 실행은 CPU와 모델 서버를 공유했다. 이 시간은 같은 조건에서 관측한 작업량이며, 일반적인 속도 순위나 청구 금액으로 해석하지 않는다.

품질 목표를 받은 그룹에서 관측한 변화

- 관측

B~F는 함수의 최대 복잡도가 5~9였고, A는 5~23이었다. 에이전트가 작성·수정한 테스트 수는 A가 실행당 평균 22.4개, B~F가 40.6개였다.
- 해석

목표를 받은 그룹은 덜 복잡한 함수와 더 많은 테스트를 남겼다. 다만 도구 안내, 수치 목표, 재측정 지시를 함께 줬으므로 무엇이 각각 얼마나 작용했는지는 분리할 수 없다.

결론 · 코드와 테스트의 품질 개선에 활용

54회 전체에서 CRAP이 낮은 실행이 기능을 더 자주 해결하는 경향을 보였다. mutation 점수와 기능 해결 사이에서는 뚜렷한 연관성을 확인하지 못했다.

그러나 CRAP·mutation 목표를 준 그룹의 기능 해결률 향상은 뚜렷하지 않았다. 목표를 주지 않은 그룹과 비교하면 모델 요청은 평균 2.6배, 작업 시간은 약 6배였다.

따라서 두 지표는 기능 해결률을 높이기 위한 목적으로만 사용하기보다, 코드의 복잡도를 낮추고 테스트 커버리지와 결함 검출력을 높여 이후의 유지보수 부담을 줄이려는 목적으로 활용할 것을 권장한다.

이번 실험에서는 실제 유지보수 작업 시간이나 수정 비용을 측정하지 않았다.